Java Lab-Test Solutions

-by Suraj Krishna Banavalikar(1MS23CS186)

1. Write a Java Program that does the following related to Inheritance:

a. **Create an abstract class called 'Vehicle' which contains the 'hashelmet','year of manufacture' and two abstract methods 'getData()' and 'putData()'. Demonstrate the error when attempt is made to create objects of 'Vehicle'.**

b. **Have two derived classes 'TwoWheeler' and 'FourWheeler'. 'FourWheeler' is a final class. Demonstrate the error when attempt is made to inherit from 'FourWheeler'.**

c. **Your abstract class should haveoverloadedconstructors that initializes 'hashelmet' and 'year of manufacture for TwoWheeler and FourWheeler respectively.**

d. **'TwoWheeler' has data elements 'Brand', 'Cost', 'EngineType' (possible values "2 stroke", "4 stroke"), and 'Color' which are private, protected, 'friendly/default' and public respectively. Demonstrate the various ways in which the two abstract methods can be dealt 'getData()' and 'putData()' can be dealt with by the derived classes, 'TwoWheeler' and 'FourWheeler'.**

Program :

```
import java.util.*;
abstract class Vehicle  //abstract class
{
    boolean hashelmet;
    int yearofmanufacture;
    Vehicle()
    {
        hashelmet = false;
        yearofmanufacture = 2000;
    }
    Vehicle(boolean h,int yom)
    {
        hashelmet = h;
        yearofmanufacture = yom;
```

```
    }
    abstract void getData();
    abstract void putData();
}
class TwoWheeler extends Vehicle
{
    private String Brand;
    protected float cost;
    String EngineType;
    public String colour;
    TwoWheeler()
    {
        super(true,2005);
    }
    void getData()
    {
        System.out.println("Two Wheeler : ");
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Brand name : ");
        Brand = sc.next();
        System.out.println("Enter cost : ");
        cost = sc.nextFloat();
        System.out.println("Enter Engine Type : ");
        EngineType = sc.next();
        System.out.println("Enter colour : ");
        colour = sc.next();
    }
    void putData()
    {
```

```
            System.out.println("\nHelmet "+hashelmet+" Manufacture "+yearofmanufacture);
            System.out.println("Brand name is "+Brand);
            System.out.println("Cost is "+cost);
            System.out.println("Engine type is "+EngineType);
            System.out.println("Colour is "+colour);
        }
}

final class FourWheeler extends Vehicle  //final class
{
        String Brand;
        float cost;
        String colour;
        FourWheeler()
        {
            super(false,2010);
        }
        void getData()
        {
            System.out.println("\n\nFour Wheeler : ");
            Scanner sca = new Scanner(System.in);
            System.out.println("Enter Brand name : ");
            Brand = sca.next();
            System.out.println("Enter cost : ");
            cost = sca.nextFloat();
            System.out.println("Enter colour : ");
            colour = sca.next();
        }
        void putData()
```

```
    {
        System.out.println("\nHelmet "+hashelmet+" Manufacture "+yearofmanufacture);
        System.out.println("Brand name is "+Brand);
        System.out.println("Cost is "+cost);
        System.out.println("Colour is "+colour);
    }
}
/*class Car extends FourWheeler  //this class cannot inherit final class Four Wheeler
{
    Car()
    {
        System.out.println("This class gives error");
    }
}*/
class VehicleMain
{
    public static void main(String[] args)
    {
        TwoWheeler TW = new TwoWheeler();
        FourWheeler FW = new FourWheeler();
        //Vehicle V = new Vehicle();  //gives error as Abstract class cannot be instantiated
        TW.getData();
        TW.putData();
        FW.getData();
        FW.putData();
    }
}
```

Output :

Two Wheeler :

Enter Brand name :

HeroHonda

Enter cost :

100000

Enter Engine Type :

2stroke

Enter colour :

Blue

Helmet true Manufacture 2005

Brand name is HeroHonda

Cost is 100000.0

Engine type is 2stroke

Colour is Blue

Four Wheeler :

Enter Brand name :

Tata

Enter cost :

500000

Enter colour :

White

Helmet false Manufacture 2010

Brand name is Tata

Cost is 500000.0

Colour is White

2. Write a Java Program that does the following

a. **Create a superclass, Student, and two subclasses, Undergrad and Grad.**

b. **The superclass Student should have the following data members: name, ID, grade, age**

c. **The superclass, Student should have at least one method: Boolean isPassed (double grade)**

d. **The purpose of the isPassed method is to take one parameter, grade (value between 0 and 100) and check whether the grade has passed the requirement for passing a course. In the Student class this method should be empty as an abstract method.**

e. **The two subclasses, Grad and Undergrad, will inherit all data members of the Student class and override the method isPassed. For the UnderGrad class, if the grade is above 70.0, then isPassedreturns true, otherwise it returns false. For the Grad class, if the grade is above 80.0, then isPassed returns true, otherwise returns false.**

f. **Demonstrate "final" keyword in the above class.**

g. **Create a test class for your three classes. In the test class, create one Grad object and one Undergrad object. For each object, provide a grade and display the results of the isPassed method.**

Program :

```
abstract class Student //abstract class student
{
    String name;
    String id;
    double grade;
    int age;
    abstract boolean isPassed(double grade); //abstract method
}
class Undergrad extends Student
{
    final String ug = "UG";  //string declared as final
    Undergrad(String name,String id,int age)
    {
            this.name = name;
            this.id = id;
```

```
            this.age = age;
    }
    boolean isPassed(double grade)
    {
            if(grade>70.0)
                    return true;
            else
                    return false;
    }
}
class Grad extends Student
{
    final String pg = "PG";  //string declared as final
    Grad(String name,String id,int age)
    {
            this.name = name;
            this.id = id;
            this.age = age;
    }
    boolean isPassed(double grade)
    {
            if(grade>80.0)
                    return true;
            else
                    return false;
    }
}
public class StudMain
{
    public static void main(String[] args)
    {
```

```
        Grad PG = new Grad("PQR","1ABC20PQR",24);

        Undergrad UG = new Undergrad("MNO","1ABC22MNO",22);  //instantiating
subclasses


        System.out.println("Graduation : "+UG.ug+"\nName is "+UG.name+"\nID :
"+UG.id+"\nAge : "+UG.age);

        System.out.println("Result Passed : "+UG.isPassed(69)+"\n");

        System.out.println("Graduation : "+PG.pg+"\nName is "+PG.name+"\nID :
"+PG.id+"\nAge : "+PG.age);

        System.out.println("Result Passed : "+PG.isPassed(89)+"\n");

    }

}
```

Output :

```
Graduation : UG

Name is MNO

ID : 1ABC22MNO

Age : 22

Result Passed : false


Graduation : PG

Name is PQR

ID : 1ABC20PQR

Age : 24

Result Passed : true
```

3. Write a Java Program that does the following

a. **Create a super class called Car. The Car class has the following fields and methods.**
    - **int speed; double regularPrice; String color; double getSalePrice();**

b. **Create a sub class of Car class and name it as Truck. The Truck class has the following fields and methods.**
    - **int weight; double getSalePrice();**
    - **//Ifweight>2000,10% discount. Otherwise,20%discount.**

c. **Create a subclass of Car class and name it as Ford. The Ford class has the following fields and methods**
    - **int year; intmanufacturerDiscount; double getSalePrice();**
    - **//From the sale price computed from Carclass, subtract the manufacturer Discount.**

d. **Create a subclass of Car class and name it as Sedan. The Sedan class has the following fields and methods.**
    - **int length; double getSalePrice();**
    - **//If length>20feet, 5% discount, Otherwise, 10% discount.**

e. **Create MyOwnAutoShop class which contains the main() method. Perform the following within the main() method.**
    - **Create an instance of Sedan class and initialize all the fields with appropriate values.**
    - **Use super(...) method in the constructor for initializing the fields of the superclass.**
    - **Create an instance of the Ford class and initialize all the fields with appropriate values**
    - **Use super(...) method in the constructor for initializing the fields of the super class.**
    - **Create an instance of Car class and initialize all the fields with appropriate values. Display the sale prices of all instances.**

Program :

```
class Car //car class
{
    int speed;
    double regularPrice;
    String color;
```

```
    Car(double rp)
    {
        regularPrice = rp;
    }
    Car(int s,String c,double rp)  //overloading constructor
    {
        speed = s;
        color = c;
        regularPrice = rp;
    }
    double getSalePrice()
    {
        return regularPrice;
    }
}
class Truck extends Car
{
    int weight;
    Truck(int w,double rp)
    {
        super(rp);
        weight = w;
    }
    double getSalePrice()
    {
        if(weight>2000)
            return regularPrice-(regularPrice*0.1);
        else
            return regularPrice-(regularPrice*0.2);
    }
}
```

```
class Ford extends Car
{
    int year;
    int manufactureDiscount;
    Ford(int y,int md,double rp)
    {
            super(rp);
            year = y;
            manufactureDiscount = md;
    }
    double getSalePrice()
    {
            return super.getSalePrice()-manufactureDiscount;
    }
}
class Sedan extends Car
{
    int length;
    Sedan(int l,double rp)
    {
            super(rp);
            length = l;
    }
    double getSalePrice()
    {
            if(length>20)
                    return regularPrice-regularPrice*0.05;
            else
                    return regularPrice-regularPrice*0.1;
    }
}
```

```
public class MyOwnAutoShop
{
    public static void main(String[] args)
    {
            Sedan S = new Sedan(21,1000000);  //instance of sedan
            Ford F = new Ford(1903,10000,10000000);  //instance of Ford
            Truck T = new Truck(2500,100000);  //instance of Truck
            Car C = new Car(200,"Blue",5000000);  //instance of car\

            System.out.println("The colour of car is "+C.color+" and speed is "+C.speed);
            System.out.println("The price of car is : "+C.getSalePrice());
            System.out.println("The price of truck is : "+T.getSalePrice());
            System.out.println("The price of Ford is : "+F.getSalePrice());
            System.out.println("The price of Sedan is : "+S.getSalePrice());
    }
}
```

Output :

```
The colour of car is Blue and speed is 200
The price of car is : 5000000.0
The price of truck is : 90000.0
The price of Ford is : 9990000.0
The price of Sedan is : 950000.0
```

4. Write a Java Program that implements the following

- **Define a class SavingsAccount with following characteristics.**
- **Use a static variable annualInterestRate to store the annual interest rate for all account holders.**
- **Private data member savingsBalance indicating the amount the saver currently has on deposit.**
- **Method calculateMonthlyInterest to calculate the monthly interest as (savingsBalance * annualInterestRate / 12). After calculation, the interest should be added to savingsBalance.**
- **Static method modifyInterestRate to set annualInterestRate.**
- **Parameterized constructor with savingsBalance as an argument to set the value of that instance.**
- **Test the class SavingsAccount to instantiate two savingsAccount objects, saver1 and saver2, with balances of Rs.2000.00 and Rs3000.00, respectively. Set annualInterestRate to 4%, then calculate the monthly interest and print the new balances for both savers. Then set the annualInterestRate to 5%, calculate the next month's interest and print the new balances for both savers.**

Program :

```
class SavingsAccount  //class savingsaccount
{
        static double annualInterestRate;  //static variable
        private double savingsBalance;
        SavingsAccount(double savingsBalance)
        {
                this.savingsBalance = savingsBalance;
        }
        double calculateMonthlyInterest()  //method to calculate monthly interest
        {
                savingsBalance += savingsBalance*annualInterestRate/12;
                return savingsBalance;
        }
        static void modifyInterestRate(double IR)
```

```
        {
                annualInterestRate = IR;
        }
}
public class Bank
{
        public static void main(String[] args)
        {
                SavingsAccount saver1 = new SavingsAccount(2000.00);
                SavingsAccount saver2 = new SavingsAccount(3000.00);

                SavingsAccount.annualInterestRate = 4;  //setting annual interest rate

                System.out.println("The balance of saver 1 in next month is : Rs.
"+saver1.calculateMonthlyInterest());
                System.out.println("The balance of saver 2 in next month is : Rs.
"+saver2.calculateMonthlyInterest());

                SavingsAccount.modifyInterestRate(5);  //modifying interest rate

                System.out.println("New balance of saver 1 in next month is : Rs.
"+saver1.calculateMonthlyInterest());
                System.out.println("New balance of saver 2 in next month is : Rs.
"+saver2.calculateMonthlyInterest());
        }
}
```

Output :

```
The balance of saver 1 in next month is : Rs. 2666.6666666666665
The balance of saver 2 in next month is : Rs. 4000.0
New balance of saver 1 in next month is : Rs. 3777.7777777777774
New balance of saver 2 in next month is : Rs. 5666.666666666667
```

5. **Write a JAVA program which does the following operations:**

a. **An Interface class for Stack Operations**

b. **A Class that implements the Stack Interface and creates a fixed length Stack.**

c. **A Class that implements the Stack Interface and creates a Dynamic Length Stack.**

A Class that uses both the above Stacks through Interface reference and does the Stack operations that demonstrates the runtime binding.

Program :

```
import java.util.Scanner;
class Stacks
{
	int top = -1;
	int max = 5;
	int arr[] = new int[max];
	void pushm(int val)
	{
		if(top == max-1)
		{
			System.out.println("Stack full ! Doubling size");
			Full();
		}
		top ++;
		arr[top] = val;
	}
	int popm()
	{
		if(top == -1)
		{
			System.out.println("Empty stack !!!");
```

```
                return 0;
            }
            int item = arr[top];
            top--;
            return item;
        }
        void disp()
        {
            if(top > -1)
            {
                System.out.println("Elements are : ");
                for(int i=0;i<=top;i++)
                    System.out.print(" "+arr[i]);
                System.out.println();
            }
            else
                System.out.println("Empty Stack !");
        }
        void Full()
        {
            int dmax = max*2;  //size doubled
            int darr[] = new int[dmax]; //new array created
            for(int i=0;i<max;i++)
                darr[i] = arr[i];  //saving data in created array
            arr = darr;      //changing reference
            max = dmax;
        }
}
public class StackMain
```

```
{
	public static void main(String[] args)
	{
		int select,value;
		Scanner sc = new Scanner(System.in);
		Stacks ST = new Stacks();
		System.out.println("Enter : \n1 for push \n2 for pop \n3 for display \n4 for
exit");
		outer:while(true)
		{
			System.out.print("Select Number : ");
			select = sc.nextInt();
			switch(select)
			{
			case 1 :
				System.out.println("Enter the element : ");
				value = sc.nextInt();
				ST.pushm(value);
				break;
			case 2 :
				value = ST.popm();
				System.out.println("Deleted elemant is : "+value);
				break;
			case 3 :
				ST.disp();
				break;
			case 4 :
				System.out.println("Exit");
				break outer;
			default :
```

```
                    System.out.println("Select proper number");
                }
            }
        }
}
```

Output :

```
Enter :
1 for push
2 for pop
3 for display
4 for exit
Select Number : 1
Enter the element :
20
Select Number : 1
Enter the element :
40
Select Number : 2
Deleted elemant is : 40
Select Number : 2
Deleted elemant is : 20
Select Number : 2
Empty stack !!!
Deleted elemant is : 0
Select Number : 1
Enter the element :
10
Select Number : 1
Enter the element :
```

20

Select Number : 1

Enter the element :

30

Select Number : 1

Enter the element :

40

Select Number : 1

Enter the element :

50

Select Number : 3

Elements are :

10 20 30 40 50

Select Number : 1

Enter the element :

60

Stack full ! Doubling size

Select Number : 1

Enter the element :

70

Select Number : 3

Elements are :

10 20 30 40 50 60 70

Select Number : 4

Exit

6. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:

a) **two instance variables x (int) and y (int).**

b) **A default (or "no-arg") constructor that construct a point at the default location of (0, 0).**

c) **A overloaded constructor that constructs a point with the given x and y coordinates.**

d) **A method setXY() to set both x and y.**

e) **A method getXY() which returns the x and y in a 2-element int array.**

f) **A toString() method that returns a string description of the instance in the format "(x, y)".**

g) **A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates.**

h) **An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)**

i) **Another overloaded distance() method that returns the distance from this point to the origin (0,0) Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.**

Program :

```
import java.lang.Math;

class MyPoint

{

        int x,y;

        MyPoint()  //default constructor

        {

                x=0;y=0;

        }

        MyPoint(int x,int y)  //parameterized constructor

        {

                this.x = x;

                this.y = y;

        }
```

```
	void setXY(int x,int y) //sets x y value
	{
		this.x = x;
		this.y = y;
	}
	int[] getXY()  //returns array reference
	{
		int TwoArr[] = {x,y};
		return TwoArr;
	}
	public String toString()
	{
		return "("+x+","+y+")";
	}
	double distance(int x,int y)  //calculates distance
	{
		double i = Math.sqrt((x - this.x)*(x - this.x)+(y - this.y)*(y - this.y));
		return i;
	}
	double distance(MyPoint another)  //calculates distance by objects
	{
		double i = Math.sqrt((another.x - x)*(another.x - x)+(another.y - y)*(another.y -
y));
		return i;
	}
	double distance()  //calculates distance from origin
	{
		double i = Math.sqrt(x*x + y*y);
		return i;
	}
```

```
}
public class TestMyPoint
{
	public static void main(String[] args)
	{
		MyPoint mp1 = new MyPoint();
		MyPoint mp2 = new MyPoint(3,4);
		int arr[] = new int[2];

		mp1.setXY(5,12);
		arr = mp1.getXY();
		System.out.println("X : "+arr[0]+" Y : "+arr[1]);

		System.out.println(mp1); //using toString function
		System.out.println(mp2);
		System.out.println("Distance between first object(5,12) and (13,27) is : "+mp1.distance(13,27));
		System.out.println("Distance between first object(5,12) and second object(3,4) is : "+mp1.distance(mp2));
		System.out.println("Distance between second object(3,4) and origin is : "+mp2.distance());
	}
}
```

Output :

```
X : 5 Y : 12
(5,12)
(3,4)
Distance between first object(5,12) and (13,27) is : 17.0
Distance between first object(5,12) and second object(3,4) is : 8.246211251235321
Distance between second object(3,4) and origin is : 5.0
```